

NLP Project

- *Ammar Yasser Abdallah*
- *22010465*

Machine Translation Using Transformers

• *Introduction*

This code implements a Transformer based sequence to sequence model for English to Spanish translation. It preprocesses a dataset of sentence pairs builds a Transformer architecture with custom encoder and decoder layers and trains the model on this data.

• *Dataset and Preprocessing*

This part of the code ensures the file is read into memory as a clean list of lines with no trailing empty strings. Each line will later be processed into English to Spanish sentence pairs.

```
[ ] text_file = "spa.txt"

[ ] with open(text_file) as f:
    lines = f.read().split("\n")[:-1]

    text_pairs = []
    for line in lines:
        eng, spa, _ = line.split("\t")
        spa = "[start] " + spa + " [end]"
        text_pairs.append((eng, spa))

[ ] for _ in range(5):
    print(random.choice(text_pairs))
```

```
random.shuffle(text_pairs)
num_val_samples = int(0.15 * len(text_pairs))
num_train_samples = len(text_pairs) - 2 * num_val_samples
train_pairs = text_pairs[:num_train_samples]
val_pairs = text_pairs[num_train_samples : num_train_samples + num_val_samples]
test_pairs = text_pairs[num_train_samples + num_val_samples :]

print(f"{len(text_pairs)} total pairs")
print(f"{len(train_pairs)} training pairs")
print(f"{len(val_pairs)} validation pairs")
print(f"{len(test_pairs)} test pairs")
```

In this part I am preparing and splitting the dataset into training, validation, and testing subsets, which is crucial for ensuring unbiased training and reliable evaluation of the model

```
def custom_standardization(input_string):
    lowercase = tf.strings.lower(input_string)
    return tf.strings.regex_replace(lowercase, "[%s]" % re.escape(strip_chars), "")

eng_vectorization = TextVectorization(
    max_tokens=vocab_size,
    output_mode="int",
    output_sequence_length=sequence_length,
)

spa_vectorization = TextVectorization(
    max_tokens=vocab_size,
    output_mode="int",
    output_sequence_length=sequence_length + 1,
    standardize=custom_standardization,
)

train_eng_texts = [pair[0] for pair in train_pairs]
train_spa_texts = [pair[1] for pair in train_pairs]
eng_vectorization.adapt(train_eng_texts)
spa_vectorization.adapt(train_spa_texts)
```

preprocesses and tokenizes English and Spanish text for a machine translation model using TensorFlow. English text is lowercased and tokenized into fixed length integer sequences, while Spanish text undergoes additional cleaning via a custom function before tokenization.

```
def format_dataset(eng, spa):
    eng = eng_vectorization(eng)
    spa = spa_vectorization(spa)
    return {
        "encoder_inputs": eng,
        "decoder_inputs": spa[:, :-1],
    },
    spa[:, 1:]

def make_dataset(pairs):
    eng_texts, spa_texts = zip(*pairs)
    eng_texts = list(eng_texts)
    spa_texts = list(spa_texts)
    dataset = tf_data.Dataset.from_tensor_slices((eng_texts, spa_texts))
    dataset = dataset.batch(batch_size)
    dataset = dataset.map(format_dataset)
    return dataset.cache().shuffle(2048).prefetch(16)

train_ds = make_dataset(train_pairs)
val_ds = make_dataset(val_pairs)
```

process tokenizes text pairs, formats them into inputs and targets for the encoder-decoder model, and optimizes data loading for efficient training.

• *Encoder:*

```
def __init__(self, embed_dim, dense_dim, num_heads, **kwargs):
    super().__init__(**kwargs)
    self.embed_dim = embed_dim
    self.dense_dim = dense_dim
    self.num_heads = num_heads
    self.attention = layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=embed_dim
    )
    self.dense_proj = keras.Sequential(
        [
            layers.Dense(dense_dim, activation="relu"),
            layers.Dense(embed_dim),
        ]
    )
    self.layernorm_1 = layers.LayerNormalization()
    self.layernorm_2 = layers.LayerNormalization()
    self.supports_masking = True

def call(self, inputs, mask=None):
    if mask is not None:
        padding_mask = ops.cast(mask[:, None, :], dtype="int32")
    else:
        padding_mask = None

    attention_output = self.attention(
        query=inputs, value=inputs, key=inputs, attention_mask=padding_mask
    )
    proj_input = self.layernorm_1(inputs + attention_output)
    proj_output = self.dense_proj(proj_input)
    return self.layernorm_2(proj_input + proj_output)

def get_config(self):
    config = super().get_config()
    config.update(
        {
            "embed_dim": self.embed_dim,
            "dense_dim": self.dense_dim,
            "num_heads": self.num_heads,
        }
    )
    return config
```

- *Embedding*

```
class PositionalEmbedding(layers.Layer):
    def __init__(self, sequence_length, vocab_size, embed_dim, **kwargs):
        super().__init__(**kwargs)
        self.token_embeddings = layers.Embedding(
            input_dim=vocab_size, output_dim=embed_dim
        )
        self.position_embeddings = layers.Embedding(
            input_dim=sequence_length, output_dim=embed_dim
        )
        self.sequence_length = sequence_length
        self.vocab_size = vocab_size
        self.embed_dim = embed_dim

    def call(self, inputs):
        length = ops.shape(inputs)[-1]
        positions = ops.arange(0, length, 1)
        embedded_tokens = self.token_embeddings(inputs)
        embedded_positions = self.position_embeddings(positions)
        return embedded_tokens + embedded_positions

    def compute_mask(self, inputs, mask=None):
        if mask is None:
            return None
        else:
            return ops.not_equal(inputs, 0)

    def get_config(self):
        config = super().get_config()
        config.update(
            {
                "sequence_length": self.sequence_length,
                "vocab_size": self.vocab_size,
                "embed_dim": self.embed_dim,
            }
        )
        return config
```

Embeddings by creating separate embeddings for tokens and their positions in a sequence. It computes the sum of token and position embeddings, allowing the model to capture both content and order of tokens

```

embed_dim = 256
latent_dim = 2048
num_heads = 8

encoder_inputs = keras.Input(shape=(None,), dtype="int64", name="encoder_inputs")
x = PositionalEmbedding(sequence_length, vocab_size, embed_dim)(encoder_inputs)
encoder_outputs = TransformerEncoder(embed_dim, latent_dim, num_heads)(x)
encoder = keras.Model(encoder_inputs, encoder_outputs)

decoder_inputs = keras.Input(shape=(None,), dtype="int64", name="decoder_inputs")
encoded_seq_inputs = keras.Input(shape=(None, embed_dim), name="decoder_state_inputs")
x = PositionalEmbedding(sequence_length, vocab_size, embed_dim)(decoder_inputs)
x = TransformerDecoder(embed_dim, latent_dim, num_heads)(x, encoded_seq_inputs)
x = layers.Dropout(0.5)(x)
decoder_outputs = layers.Dense(vocab_size, activation="softmax")(x)
decoder = keras.Model([decoder_inputs, encoded_seq_inputs], decoder_outputs)

decoder_outputs = decoder([decoder_inputs, encoder_outputs])
transformer = keras.Model(
    [encoder_inputs, decoder_inputs], decoder_outputs, name="transformer"
)

```

Encoder processes the input sequence and the decoder generates the output sequence, with both parts incorporating positional embeddings and transformer layers for sequence-to-sequence tasks like machine translation.

• *Train model*

```

epochs = 10

transformer.summary()
transformer.compile(
    "rmsprop", loss="sparse_categorical_crossentropy", metrics=["accuracy"]
)
transformer.fit(train_ds, epochs=epochs, validation_data=val_ds)

```

Layer (type)	Output Shape	Param #	Connected to
encoder_inputs (InputLayer)	(None, None)	0	-
positional_embedding (PositionalEmbedding)	(None, None, 256)	3,045,120	encoder_inputs[0][]
decoder_inputs (InputLayer)	(None, None)	0	-
transformer_encoder (TransformerEncoder)	(None, None, 256)	3,155,456	positional_embedding[...]
functional_3 (Functional)	(None, None, 150000)	12,959,640	decoder_inputs[0][], transformer_encoder[...]

Total params: 19,920,434 (152.28 MB)
 Trainable params: 19,900,416 (76.14 MB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 19,900,416 (76.14 MB)

Epoch 1/10
 775/775 — 182s 116ms/step - accuracy: 0.7282 - loss: 1.7819 - val_accuracy: 0.7787 - val_loss: 1.3914
 Epoch 2/10
 775/775 — 122s 97ms/step - accuracy: 0.7678 - loss: 1.4542 - val_accuracy: 0.9291 - val_loss: 0.5316
 Epoch 3/10
 775/775 — 82s 98ms/step - accuracy: 0.9033 - loss: 0.6971 - val_accuracy: 0.7363 - val_loss: 1.5042
 Epoch 4/10
 775/775 — 83s 99ms/step - accuracy: 0.7291 - loss: 1.5546 - val_accuracy: 0.7441 - val_loss: 1.4157
 Epoch 5/10
 775/775 — 81s 98ms/step - accuracy: 0.9015 - loss: 0.6283 - val_accuracy: 0.9819 - val_loss: 0.1559
 Epoch 6/10
 775/775 — 80s 95ms/step - accuracy: 0.9794 - loss: 0.1596 - val_accuracy: 0.9899 - val_loss: 0.0929
 Epoch 7/10
 775/775 — 73s 94ms/step - accuracy: 0.9869 - loss: 0.1044 - val_accuracy: 0.9836 - val_loss: 0.1233
 Epoch 8/10
 775/775 — 74s 96ms/step - accuracy: 0.9887 - loss: 0.0992 - val_accuracy: 0.9965 - val_loss: 0.0404
 Epoch 9/10
 775/775 — 73s 94ms/step - accuracy: 0.9960 - loss: 0.0394 - val_accuracy: 0.9979 - val_loss: 0.0274
 Epoch 10/10
 775/775 — 73s 94ms/step - accuracy: 0.9976 - loss: 0.0295 - val_accuracy: 0.9989 - val_loss: 0.0191

The model is trained for 10 epochs, with metrics logged for accuracy and loss on both training and validation datasets. Training and validation datasets are batched, shuffled, and pre-fetched for performance.

- *Decode sequence*

```
spa_vocab = spa_vectorization.get_vocabulary()
spa_index_lookup = dict(zip(range(len(spa_vocab)), spa_vocab))
max_decoded_sentence_length = 20

def decode_sequence(input_sentence):
    tokenized_input_sentence = eng_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = spa_vectorization([decoded_sentence])[:, :-1]
        predictions = transformer([tokenized_input_sentence, tokenized_target_sentence])

        # ops.argmax(predictions[0, i, :]) is not a concrete value for jax here
        sampled_token_index = ops.convert_to_numpy(
            ops.argmax(predictions[0, i, :])
        ).item(0)
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token

        if sampled_token == "[end]":
            break
    return decoded_sentence

test_eng_texts = [pair[0] for pair in test_pairs]
for _ in range(10):
    input_sentence = random.choice(test_eng_texts)
    translated = decode_sequence(input_sentence)
    print(input_sentence)
    print(translated)
    print('=====')
```

```
Input: Do you hear me?
Translated: [start] ¿Me oyes? [end]
=====
Input: I need to get to work.
Translated: [start] Necesito ir a trabajar. [end]
=====
Input: The phone stopped working.
Translated: [start] El teléfono dejó de funcionar. [end]
=====
Input: Tom came and sat down next to Mary.
Translated: [start] Tom vino y se sentó al lado de Mary. [end]
=====
Input: Give me some time to think.
Translated: [start] Dame un poco de tiempo para pensar. [end]
=====
Input: She must be ill in bed.
Translated: [start] Debe de estar en la cama enfermo. [end]
=====
Input: Wisdom does not automatically come with age.
Translated: [start] La sabiduría no viene automáticamente con los años. [end]
=====
Input: I was on the point of leaving home when a light rain started to fall.
Translated: [start] Estaba a punto de dejar la casa cuando empezó a caer una ligera lluvia. [end]
=====
Input: Tom has been traveling all year.
Translated: [start] Tom ha estado viajando todo el año. [end]
=====
Input: Tom was the best by far.
Translated: [start] Tom fue el mejor de lejos. [end]
```

Pretrained Transformer Model GPT-2

- *Introduction*

GPT-2 model for English-Spanish machine translation using a dataset of sentence pairs. It processes the data, tokenizes the sentences, and trains the model with custom loss tracking. After training, the model is saved and used for generating translations

- *Data cleaning*

```
# Remove unnecessary whitespace and special characters
df["english"] = df["english"].str.strip().replace(r"[^a-zA-Z\s]", "", regex=True)
df["spanish"] = df["spanish"].str.strip().replace(r"[^a-zA-Z\s]", "", regex=True)

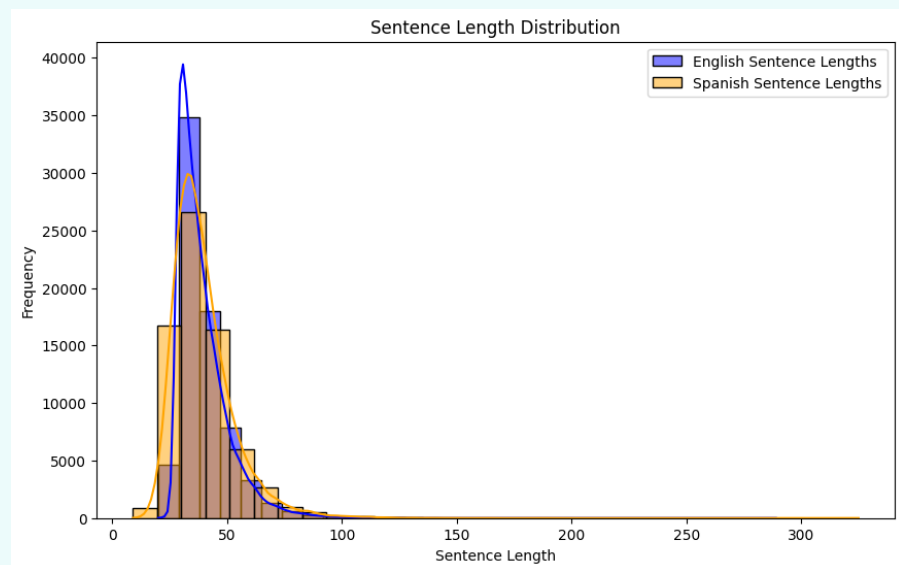
# Check for null or empty entries
df.dropna(inplace=True)
df = df[df["english"] != ""]
df = df[df["spanish"] != ""]

# Convert to Hugging Face Dataset
dataset = Dataset.from_pandas(df)
```

cleans the English and Spanish text by removing extra whitespace and non-alphabetical characters. It then drops rows with missing or empty entries

- *Visualization*

distribution of sentence lengths for both English and Spanish sentences in the dataset.



• *Tokenizer*

```
from transformers import GPT2Tokenizer

# Load the GPT-2 tokenizer
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

# Add padding token if missing
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# Tokenize the dataset
def tokenize_function(examples):
    tokenized = tokenizer(
        examples["english"], # Tokenize English sentences
        padding="max_length",
        truncation=True,
        max_length=50,
    )
    tokenized["labels"] = tokenized["input_ids"].copy()
    return tokenized

# Tokenize dataset
tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

loads the GPT-2 tokenizer and sets the padding token if it's missing. It defines a tokenize function to tokenize the English sentences, with padding, truncation, and a maximum length of 50. The dataset is then tokenized in batches using the map method

• *Train model*

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    tokenizer=tokenizer,
```

[1668/1668 34:45, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	0.417500	0.458382
2	0.416200	0.457189
3	0.402100	0.457183

training process. It initializes with the model, training arguments, training and validation datasets, and the tokenizer. This setup handles the training loop, evaluation, and logging automatically

- *Fine-Tuned Model*

```
from transformers import pipeline

# Load the fine-tuned model
generator = pipeline("text-generation", model="./fine_tuned_gpt2", tokenizer=tokenizer)

# Generate text
input_text = " cat is "
generated_text = generator(input_text, max_length=50, num_return_sequences=1)
print(generated_text[0]["generated_text"])

Device set to use cuda:0
cat is centimeters in length
```

Hugging Face API load a fine-tuned GPT-2 model for text generation. It generates text starting from the input with a maximum length of 50 tokens, and outputs the generated text.

RNN Machine Translation

- *Train model*

```
model = tf.keras.Model([encoder_inputs, decoder_inputs], decoder_outputs)

# Compile the Model
model.compile(optimizer=Adam(), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Summary of the Model
model.summary()

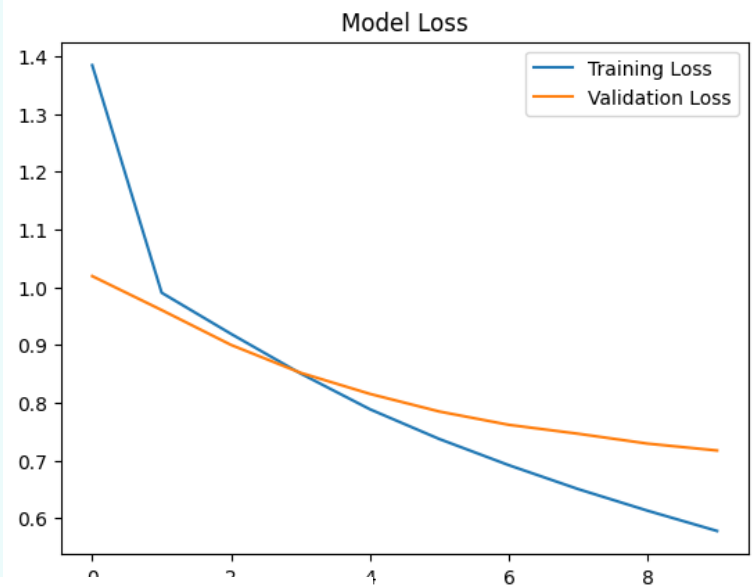
# Train the model with the padded data
model.fit([english_sentences, decoder_input_data], decoder_target_data, batch_size=64, epochs=10, validation_split=0.2)
```

The code builds a sequence-to-sequence model using an encoder-decoder architecture, compiling it with the Adam optimizer and sparse categorical cross-entropy loss. It then prints a summary of the model's structure

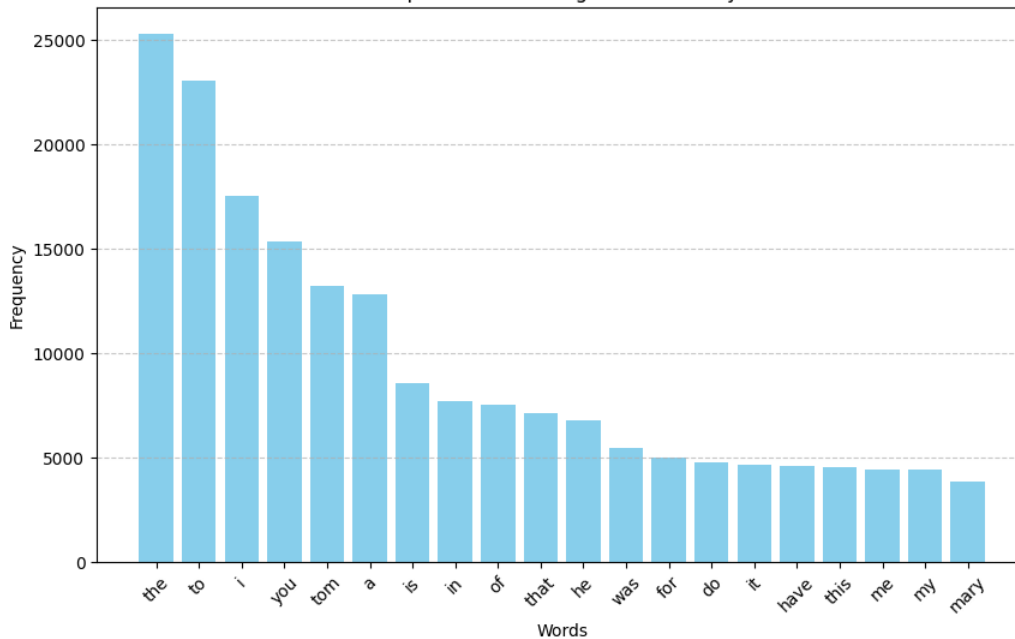
```
Total params: 74,128 (290.33 KB)
Trainable params: 74,128 (290.33 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
1724/1724 ————— 65s 27ms/step - accuracy: 0.8546 - loss: 0.8628 - val_accuracy: 0.9335 - val_loss: 0.2077
Epoch 2/10
1724/1724 ————— 15s 9ms/step - accuracy: 0.9377 - loss: 0.1890 - val_accuracy: 0.9477 - val_loss: 0.1481
Epoch 3/10
1724/1724 ————— 15s 9ms/step - accuracy: 0.9516 - loss: 0.1376 - val_accuracy: 0.9573 - val_loss: 0.1195
Epoch 4/10
1724/1724 ————— 15s 9ms/step - accuracy: 0.9594 - loss: 0.1137 - val_accuracy: 0.9625 - val_loss: 0.1043
Epoch 5/10
1724/1724 ————— 15s 9ms/step - accuracy: 0.9628 - loss: 0.1016 - val_accuracy: 0.9628 - val_loss: 0.1003
Epoch 6/10
1724/1724 ————— 15s 9ms/step - accuracy: 0.9647 - loss: 0.0949 - val_accuracy: 0.9633 - val_loss: 0.0982
Epoch 7/10
1724/1724 ————— 15s 9ms/step - accuracy: 0.9655 - loss: 0.0918 - val_accuracy: 0.9661 - val_loss: 0.0903
Epoch 8/10
1724/1724 ————— 15s 9ms/step - accuracy: 0.9667 - loss: 0.0879 - val_accuracy: 0.9648 - val_loss: 0.0942
Epoch 9/10
1724/1724 ————— 15s 9ms/step - accuracy: 0.9677 - loss: 0.0854 - val_accuracy: 0.9676 - val_loss: 0.0867
Epoch 10/10
1724/1724 ————— 15s 9ms/step - accuracy: 0.9695 - loss: 0.0813 - val_accuracy: 0.9700 - val_loss: 0.0801
```

• Visualization

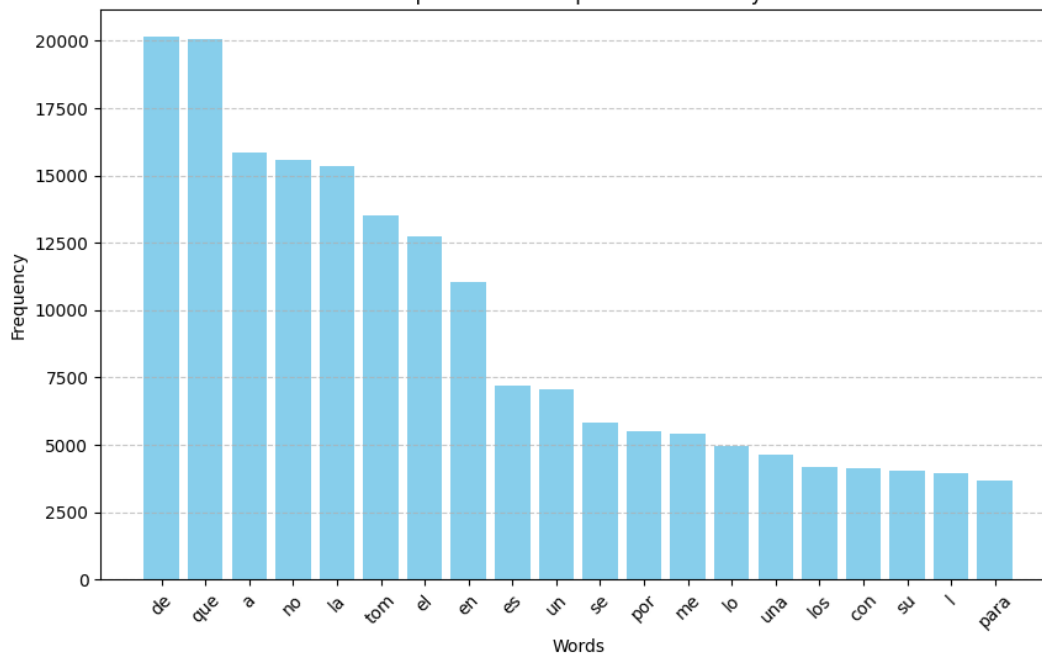
display a graph comparing the training loss and validation loss to help assess how well the model is learning and whether it is overfitting or underfitting.



Top 20 Words in English Vocabulary



Top 20 Words in Spanish Vocabulary



Ammar Yasser Abdallah

22010465